Operating Systems: Processes, Threads and CPU Scheduling

Study notes for an introductory operating systems course.

1. Programs and processes

A program is a passive entity: a file of instructions stored on disk. A process is a program in execution. When the operating system loads a program into memory and starts running it, the program becomes a process with its own state, resources and place in the scheduler. The same program can run as several independent processes at once.

Each process has its own address space, which is divided into several regions:

- The text (code) section contains the executable instructions.
- The data section stores global and static variables.
- The heap provides memory that is allocated dynamically while the program runs.
- The stack keeps function parameters, return addresses and local variables, and grows and shrinks with each call.

The operating system gives every process a unique process identifier (PID). Keeping processes in separate address spaces is what lets one crashing program fail without corrupting the memory of the others.

2. Process states

As it runs, a process moves between five states:

- New: the process is being created.
- Ready: the process is waiting to be assigned to a CPU.
- Running: its instructions are being executed on a CPU.
- Waiting (blocked): the process is waiting for an event, such as an I/O operation completing.
- Terminated: the process has finished execution.

Only one process can be running on each CPU core at any instant, but many processes can be ready or waiting. A running process moves to the waiting state when it requests I/O, and returns to the ready state when the I/O finishes. A timer interrupt can also move a running process back to the ready state so that another process gets a turn.

3. The process control block and context switching

The operating system represents each process with a data structure called the process control block (PCB). The PCB stores the process state, the program counter, the CPU registers, scheduling information such as priority, memory-management information, accounting information, and the list of open files.

Switching the CPU from one process to another is called a context switch. The kernel saves the state of the old process into its PCB and loads the saved state of the new process from its PCB. A

context switch is pure overhead: the system does no useful work for either process while it happens, so its cost depends heavily on hardware support.

4. Creating and ending processes

On UNIX-like systems a new process is created with the fork() system call. The child process receives a copy of the parent's address space, and fork() returns 0 in the child and the child's PID in the parent. The exec() family of calls then replaces the child's memory image with a new program. A parent can call wait() to pause until a child finishes and to collect the child's exit status.

A process that has terminated but whose parent has not yet called wait() is called a zombie: its PCB entry remains so that the exit status can still be collected. If a parent terminates without waiting, its children become orphans; on Linux they are adopted by the init process (PID 1), which periodically calls wait() to clean them up.

5. Threads

A thread is the basic unit of CPU use within a process. Threads of the same process share its code, data, heap and open files, but each thread has its own program counter, registers and stack.

Multithreading brings four main benefits:

- Responsiveness: an application can keep responding while one thread is blocked.
- Resource sharing: threads share memory without needing explicit inter-process communication.
- Economy: creating a thread and switching between threads is cheaper than doing the same for processes.
- Scalability: threads of one process can run in parallel on multiple cores.

Because threads share memory, two threads that update the same variable without synchronisation can produce a race condition, where the result depends on the exact order in which the threads run. Locks such as mutexes prevent this by allowing only one thread at a time into a critical section.

6. CPU scheduling

The CPU scheduler chooses which ready process runs next. Scheduling is non-preemptive when a process keeps the CPU until it blocks or finishes, and preemptive when the operating system can take the CPU away, for example when a time slice expires.

Schedulers are compared using several criteria:

- CPU utilisation: the fraction of time the CPU is busy.
- Throughput: the number of processes completed per unit of time.
- Turnaround time: the time from a process's arrival to its completion.
- Waiting time: the total time a process spends in the ready queue.
- Response time: the time from a request until the first response is produced.

First-Come, First-Served (FCFS) runs processes in arrival order. It is simple but suffers from the convoy effect, where short processes wait behind one long process. Shortest-Job-First (SJF) runs the process with the shortest next CPU burst and gives the minimum average waiting time for a given set of processes; its preemptive version is called Shortest-Remaining-Time-First (SRTF). Round Robin (RR) gives each process a fixed time quantum in turn and is designed for time-sharing systems. Priority scheduling runs the highest-priority process first and can cause starvation of low-priority processes, which is solved by aging: gradually raising the priority of processes that have waited a long time.

7. Worked example: comparing schedulers

Four processes arrive at time 0 in the order P1, P2, P3, P4, with the following CPU burst times in milliseconds:

Process	Burst time (ms)
P1	8
P2	4
P3	9
P4	5

With FCFS the processes run in the order P1, P2, P3, P4. Their waiting times are 0, 8, 12 and 21 ms, so the average waiting time is 41 / 4 = 10.25 ms. Their completion times are 8, 12, 21 and 26 ms, so the average turnaround time is 67 / 4 = 16.75 ms.

With SJF the order becomes P2, P4, P1, P3. The waiting times are P2 = 0, P4 = 4, P1 = 9 and P3 = 17 ms, giving an average waiting time of 30 / 4 = 7.5 ms, and the average turnaround time falls to 56 / 4 = 14 ms.

With Round Robin and a time quantum of 4 ms, P2 finishes at 8 ms, P1 at 20 ms, P4 at 25 ms and P3 at 26 ms. The waiting times are P1 = 12, P2 = 4, P3 = 17 and P4 = 20 ms, so the average waiting time is 53 / 4 = 13.25 ms. Round Robin has a higher average waiting time than SJF here, but no process waits more than three quanta for its first turn, which gives better response time.

The choice of quantum matters: if it is very large, Round Robin behaves like FCFS; if it is very small, the system spends a large share of its time on context switches.